CURO MEDICAL MANAGEMENT SYSTEM • PRODUCTION ENGINEERING

Complete Step-by-Step Database Implementation Guide

An easy-to-follow, comprehensive engineering manual for designing, configuring, creating, seeding, and integrating the production relational database for the CURO Medical System.

System Edition: CURO Medical Suite 4.0	**Recommended DBMS:** PostgreSQL 15+ / 16 (or MySQL 8.0)
Inventory Strategy: FEFO (First-Expire, First-Out)	**Compliance:** Audit Ledger, RBAC & WHO-GPP Aligned

8-Step Implementation Roadmap

- **Step 1: Choose Your Database Engine** — Pros and cons of PostgreSQL, MySQL, and SQLite.
- **Step 2: 5-Minute Installation & pgAdmin Setup** — Simple walkthrough to get the database running.
- **Step 3: Database Blueprint & The 6 Core Domains** — Understanding the tables and how they link together.
- **Step 4: Execute DDL Table Creation (Ordered Sequence)** — Copy-paste SQL scripts with foreign keys.
- **Step 5: Seed Master Data & Test Credentials** — Pre-populating default roles, admin user, medicines, suppliers.
- **Step 6: Master FEFO & Alert Queries** — Exact SQL logic powering barcode checkout and stock warnings.
- **Step 7: Build Node.js / Express API** — Connecting the frontend and executing atomic sales transactions.
- **Step 8: Quality Assurance & Production Verification** — Integrity checklist, backup routines, and common fixes.

Step 1: Choose Your Database Engine

A healthcare pharmacy system requires strict **ACID** (Atomicity, Consistency, Isolation, Durability) transactions so that medicine stock numbers are never corrupted during simultaneous checkouts.

Engine	Score & Tier	Why & When to Select It
PostgreSQL 15+ ***(Recommended)***	Enterprise Grade ★★★★★	Best choice for CURO. Free, open-source, robust concurrency locks, native JSON for prescriptions, and pgAdmin graphical management tool.
MySQL 8.0+	Web Standard ★★★★■	Great option if your web hosting already features cPanel, phpMyAdmin, or XAMPP. Full schema compatibility.
SQLite 3	Local / Portable ★★★■■	Zero installation. Stored in a single file on disk. Ideal if building a standalone desktop application with Electron.

Step 2: 5-Minute PostgreSQL & pgAdmin Setup

Follow these 5 simple steps to install and prepare your database:

1. **Download:** Go to https://www.postgresql.org/download/windows/ and download the Windows installer (v16 or v15).
2. **Run Installation:** Launch the installer. Leave all default components selected (PostgreSQL Server, pgAdmin 4, Command Line Tools).
3. **Define Superuser Password:** Enter a secure password for the postgres user (e.g. root1234). *Make sure to write it down!*
4. **Complete Wizard:** Accept the default port 5432 and default locale. Click Next to finish.
5. **Create Database in pgAdmin:** Launch **pgAdmin 4** from the Start menu. Under **Servers** → right-click **Databases** → select **Create > Database...**. Type curo_medical_db and click **Save**.

HOW TO EXECUTE SQL IN PGADMIN: To run any of the SQL scripts in this manual, right-click on **curo_medical_db** in pgAdmin and select **Query Tool**. Paste the script and click the **Execute (Play ■)** button.

Step 3: Database Blueprint — The 6 Core Domains

The system is organized into 6 domains for clean separation of concerns:

Domain	Tables	Clinical & Operational Responsibilities
1. Security & RBAC	roles users	Manages user accounts, encrypted passwords, roles (Admin, Pharmacist, Cashier, Doctor, Inventory).
2. Drug Master	categories manufacturers medicines	Catalog of brand names, generics, strengths, barcode numbers, reorder thresholds.
3. Inventory & FEFO	suppliers batches inventory_movements	Expiry dates, batch lots, cost and retail prices, shelf locations, and full stock movement history.
4. Clinical & Patient	customers doctors prescriptions prescription_items	Patient credit limits, doctor medical licenses, and prescription-to-POS dispensing.
5. Procurement	purchase_orders purchase_order_items purchases	Supplier purchase orders, goods receipt notes, and supplier ledger dues.
6. POS & Financials	sales sale_items payments returns audit_logs	Cash register sales, split tender payments, returns, and regulatory audit compliance.

Step 4: Execute Table Creation (DDL Scripts)

Important Rule: Tables must be executed in order so that Foreign Keys resolve correctly. Run these 4 modular scripts sequentially:

Section A: Security, Categories & Suppliers

```
CREATE TABLE IF NOT EXISTS roles (
    id VARCHAR(32) PRIMARY KEY,
    name VARCHAR(64) NOT NULL,
    description VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS users (
    id VARCHAR(36) PRIMARY KEY,
    username VARCHAR(64) NOT NULL UNIQUE,
    email VARCHAR(128) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    name VARCHAR(128) NOT NULL,
    designation VARCHAR(128) NOT NULL,
    role_id VARCHAR(32) NOT NULL REFERENCES roles(id) ON UPDATE CASCADE,
    phone VARCHAR(32),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS categories (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(128) NOT NULL UNIQUE,
    parent_id VARCHAR(36) REFERENCES categories(id) ON DELETE SET NULL,
    description TEXT
);

CREATE TABLE IF NOT EXISTS manufacturers (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE,
    contact_info TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS suppliers (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    contact_person VARCHAR(100),
    phone VARCHAR(32) NOT NULL,
    email VARCHAR(128),
    address TEXT,
    tax_number VARCHAR(64),
    credit_period_days INT DEFAULT 30,
    outstanding_balance DECIMAL(12,2) DEFAULT 0.00,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Section B: Medicines & FEFO Batches

```
CREATE TABLE IF NOT EXISTS medicines (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    generic_name VARCHAR(150) NOT NULL,
    category_id VARCHAR(36) NOT NULL REFERENCES categories(id),
    manufacturer_id VARCHAR(36) NOT NULL REFERENCES manufacturers(id),
    composition VARCHAR(255),
    dosage_form VARCHAR(50) NOT NULL,
    strength VARCHAR(50),
    barcode VARCHAR(64) UNIQUE,
    hsn_code VARCHAR(32),
    requires_prescription BOOLEAN DEFAULT FALSE,
    reorder_level INT DEFAULT 50,
    reorder_quantity INT DEFAULT 100,
    rack_location VARCHAR(64),
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS batches (
    id VARCHAR(36) PRIMARY KEY,
    medicine_id VARCHAR(36) NOT NULL REFERENCES medicines(id),
    supplier_id VARCHAR(36) NOT NULL REFERENCES suppliers(id),
    batch_number VARCHAR(64) NOT NULL,
    manufacture_date DATE NOT NULL,
    expiry_date DATE NOT NULL,
    cost_price DECIMAL(10,2) NOT NULL CHECK (cost_price >= 0),
    selling_price DECIMAL(10,2) NOT NULL CHECK (selling_price >= 0),
    mrp DECIMAL(10,2) NOT NULL CHECK (mrp >= selling_price),
    quantity_received INT NOT NULL CHECK (quantity_received >= 0),
    quantity_available INT NOT NULL CHECK (quantity_available >= 0),
    rack_location VARCHAR(64),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Section C: Clinical & Prescriptions

```
CREATE TABLE IF NOT EXISTS customers (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(128) NOT NULL,
    phone VARCHAR(32) NOT NULL UNIQUE,
    email VARCHAR(128),
    address TEXT,
    date_of_birth DATE,
    credit_limit DECIMAL(10,2) DEFAULT 0.00,
    outstanding_balance DECIMAL(10,2) DEFAULT 0.00,
    loyalty_points INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS doctors (
    id VARCHAR(36) PRIMARY KEY,
    name VARCHAR(128) NOT NULL,
    specialization VARCHAR(128) NOT NULL,
    license_number VARCHAR(64) NOT NULL UNIQUE,
    phone VARCHAR(32),
    chamber VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS prescriptions (
    id VARCHAR(36) PRIMARY KEY,
    customer_id VARCHAR(36) NOT NULL REFERENCES customers(id),
    doctor_id VARCHAR(36) NOT NULL REFERENCES doctors(id),
    prescription_date DATE NOT NULL,
    notes TEXT,
    status VARCHAR(32) DEFAULT 'Pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS prescription_items (
    id VARCHAR(36) PRIMARY KEY,
    prescription_id VARCHAR(36) NOT NULL REFERENCES prescriptions(id) ON DELETE CASCADE,
    medicine_id VARCHAR(36) NOT NULL REFERENCES medicines(id),
    dosage VARCHAR(64) NOT NULL,
    duration_days INT NOT NULL,
    quantity_prescribed INT NOT NULL
);
```

Section D: POS Sales, Items & Audit Ledger

```
CREATE TABLE IF NOT EXISTS sales (
    id VARCHAR(36) PRIMARY KEY,
    invoice_number VARCHAR(64) NOT NULL UNIQUE,
    customer_id VARCHAR(36) REFERENCES customers(id),
    prescription_id VARCHAR(36) REFERENCES prescriptions(id),
    subtotal DECIMAL(10,2) NOT NULL,
    discount_amount DECIMAL(10,2) DEFAULT 0.00,
    tax_amount DECIMAL(10,2) DEFAULT 0.00,
    total_amount DECIMAL(10,2) NOT NULL,
    payment_mode VARCHAR(32) NOT NULL,
    payment_status VARCHAR(32) NOT NULL,
    served_by VARCHAR(36) NOT NULL REFERENCES users(id),
    sale_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS sale_items (
    id VARCHAR(36) PRIMARY KEY,
    sale_id VARCHAR(36) NOT NULL REFERENCES sales(id) ON DELETE CASCADE,
    medicine_id VARCHAR(36) NOT NULL REFERENCES medicines(id),
    batch_id VARCHAR(36) NOT NULL REFERENCES batches(id),
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL,
    subtotal DECIMAL(10,2) NOT NULL
);

CREATE TABLE IF NOT EXISTS inventory_movements (
    id VARCHAR(36) PRIMARY KEY,
    medicine_id VARCHAR(36) NOT NULL REFERENCES medicines(id),
    batch_id VARCHAR(36) NOT NULL REFERENCES batches(id),
    movement_type VARCHAR(32) NOT NULL, -- PURCHASE, SALE, RETURN, ADJUSTMENT
    quantity INT NOT NULL,
    balance_after INT NOT NULL,
    reference_type VARCHAR(32) NOT NULL, -- PO, INV, MANUAL
    reference_id VARCHAR(64) NOT NULL,
    user_id VARCHAR(36) REFERENCES users(id),
    notes VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS audit_logs (
    id VARCHAR(36) PRIMARY KEY,
    user_id VARCHAR(36) REFERENCES users(id),
    user_name VARCHAR(128) NOT NULL,
    action VARCHAR(64) NOT NULL,
    table_name VARCHAR(64) NOT NULL,
    record_id VARCHAR(64) NOT NULL,
    details TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- PERFORMANCE SPEED INDEXES
CREATE INDEX IF NOT EXISTS idx_med_barcode ON medicines(barcode);
CREATE INDEX IF NOT EXISTS idx_batch_fefo ON batches(medicine_id, expiry_date, quantity_available);
CREATE INDEX IF NOT EXISTS idx_sales_inv ON sales(invoice_number);
CREATE INDEX IF NOT EXISTS idx_cust_phone ON customers(phone);
```

Step 5: Seed Master & Demo Data

Run this seed script to populate default system roles, an administrator account, medicine categories, and sample suppliers:

```
-- 1. Default Roles
INSERT INTO roles (id, name, description) VALUES
('admin', 'Administrator', 'Full system control, financials, user management'),
('pharmacist', 'Pharmacist', 'Dispense medicines, verify prescriptions, manage inventory'),
('cashier', 'Cashier', 'Process POS sales, invoices, customer credit collection'),
('inventory_manager', 'Inventory Manager', 'Purchase orders, goods receipt, batch control'),
('doctor', 'Doctor / Staff', 'Write prescriptions, view clinical medicine catalog')
ON CONFLICT (id) DO NOTHING;

-- 2. Default Administrator (Username: admin / Password: root)
INSERT INTO users (id, username, email, password_hash, name, designation, role_id, phone, is_active)
VALUES (
    'usr-1', 'admin', 'pg@gmail.com',
    '$2a$10$wK1y9Bv4g9N4qC8Q1tQ6yO3fR6pQ0v8r5Y8k0N4qC8Q1tQ6yO3fR6',
    'Dr. Alexander Vance', 'Chief Administrative Officer', 'admin', '+880 1711-000111', TRUE
) ON CONFLICT (id) DO NOTHING;

-- 3. Essential Drug Categories
INSERT INTO categories (id, name, description) VALUES
('cat-1', 'Analgesics & Pain Relief', 'Pain relievers and antipyretics'),
('cat-2', 'Antibiotics & Antimicrobials', 'Bacterial infection treatments'),
('cat-3', 'Antacids & Gastrointestinal', 'Heartburn, ulcer and digestive care'),
('cat-6', 'Diabetic Care', 'Blood glucose management and insulin')
ON CONFLICT (id) DO NOTHING;

-- 4. Sample Manufacturers
INSERT INTO manufacturers (id, name, contact_info) VALUES
('mfr-1', 'Square Pharmaceuticals PLC', 'dhaka@squarepharma.com.bd | +880-2-8833047'),
('mfr-2', 'Beximco Pharmaceuticals Ltd', 'info@bpl.net | +880-2-58611001')
ON CONFLICT (id) DO NOTHING;

-- 5. Sample Suppliers
INSERT INTO suppliers (id, name, contact_person, phone, email, credit_period_days, outstanding_balance) VALUES
('sup-1', 'Square Distribution Logistics', 'Asif Iqbal', '+880 1711-223344', 'dist@square.com', 30, 45200.00),
('sup-2', 'Beximco Prime Distributors', 'Tanvir Ahmed', '+880 1819-334455', 'supply@beximco.com', 45, 32000.00)
ON CONFLICT (id) DO NOTHING;
```

Step 6: Master FEFO & Critical Alert Queries

FEFO (First Expire, First Out) ensures that medicines nearing expiration are sold before newer stock. Below are the 3 essential queries running in CURO:

1. FEFO Batch Picker for POS Checkout:

```
SELECT id, batch_number, expiry_date, quantity_available, selling_price
FROM batches
WHERE medicine_id = 'med-1'
  AND quantity_available > 0
  AND expiry_date > CURRENT_DATE
ORDER BY expiry_date ASC;
```

2. Near-Expiry Alert Query (< 45 Days Remaining):

```
SELECT m.name, b.batch_number, b.expiry_date, b.quantity_available,
       (b.expiry_date - CURRENT_DATE) AS days_remaining
FROM batches b
JOIN medicines m ON m.id = b.medicine_id
WHERE b.quantity_available > 0
  AND b.expiry_date BETWEEN CURRENT_DATE AND (CURRENT_DATE + INTERVAL '45 days')
ORDER BY b.expiry_date ASC;
```

3. Low Stock Alert Query (Below Reorder Threshold):

```
SELECT m.id, m.name, m.reorder_level,
       COALESCE(SUM(b.quantity_available), 0) AS current_stock
FROM medicines m
LEFT JOIN batches b ON b.medicine_id = m.id AND b.expiry_date > CURRENT_DATE
WHERE m.is_active = TRUE
GROUP BY m.id, m.name, m.reorder_level
HAVING COALESCE(SUM(b.quantity_available), 0) <= m.reorder_level;
```

Step 7: Connect Frontend to Database (Node.js API)

To replace the current client-side aps-store.js with your live PostgreSQL database, follow this straightforward 4-step backend pattern:

1. **Install Backend Packages:** In your project directory, run:
   npm install express pg cors dotenv jsonwebtoken bcryptjs
2. **Configure Connection:** In a new file db.js, configure the connection pool:

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  user: process.env.DB_USER || 'postgres',
  host: process.env.DB_HOST || 'localhost',
  database: process.env.DB_NAME || 'curo_medical_db',
  password: process.env.DB_PASSWORD || 'root1234',
  port: process.env.DB_PORT || 5432,
  max: 20 // Handles up to 20 concurrent terminal connections
});

module.exports = pool;
```

3. Atomic POS Checkout Endpoint with ACID Protection:

A sale must be processed inside a database transaction so that stock updates and invoice creation either both succeed or both rollback:

```
const express = require('express');
const router = express.Router();
const pool = require('./db');

router.post('/checkout', async (req, res) => {
  const client = await pool.connect();
  try {
    await client.query('BEGIN'); // Start ACID Transaction
    const { invoiceNumber, customerId, items, subtotal, totalAmount, paymentMode, servedBy } = req.body;
    const saleId = 'sale_' + Date.now();

    // 1. Insert Sales Record
    await client.query(
      `INSERT INTO sales (id, invoice_number, customer_id, subtotal, total_amount, payment_mode, payment_status,
 served_by)
       VALUES ($1, $2, $3, $4, $5, $6, 'Paid', $7)`,
      [saleId, invoiceNumber, customerId || null, subtotal, totalAmount, paymentMode, servedBy]
    );

    // 2. Loop Items: Insert Sale Items & Deduct FEFO Batch Stock
    for (const item of items) {
      await client.query(
        `INSERT INTO sale_items (id, sale_id, medicine_id, batch_id, quantity, unit_price, subtotal)
         VALUES ($1, $2, $3, $4, $5, $6, $7)`,
        ['si_' + Math.random().toString(36).substr(2,9), saleId, item.medicineId, item.batchId, item.quantity, i
tem.unitPrice, item.subtotal]
      );

      // Decrement Available Batch Stock
      await client.query(
        `UPDATE batches SET quantity_available = quantity_available - $1 WHERE id = $2`,
        [item.quantity, item.batchId]
      );
    }

    await client.query('COMMIT'); // Commit Everything
    res.json({ success: true, invoiceNumber: invoiceNumber });
  } catch (err) {
    await client.query('ROLLBACK'); // Cancel on Error
    res.status(500).json({ error: err.message });
  } finally {
    client.release();
  }
});

module.exports = router;
```

Step 8: Production Checklist & Verification

Before deploying to active cashier terminals, verify your database against this checklist:

Verification Area	Pass Condition	Standard Fix
Stock Integrity	quantity_available never goes negative.	Ensure CHECK (quantity_available >= 0) exists on batches table.
Data Preservation	Past invoices cannot be deleted if a medicine is discontinued.	Use is_active = FALSE (soft-delete). Never perform hard SQL DELETE on medicines.
Backup Routine	Automated daily database dumps created.	Run daily cron/Task Scheduler: pg_dump -U postgres curo_medical_db > curo_backup.sql
Barcode Index	Scanner resolves drug info in under 15ms.	Ensure idx_med_barcode index is created on medicines(barcode).

Quality Assurance: Verified & Validated Twice Against CURO Architecture	**Version:** 4.0.0 Production Manual

Compliance Standard: WHO Good Pharmacy Practice (GPP) Compatible	**Author:** CURO Medical Engineering Core